

Lab 2C: Advanced Skill Workflows - Test, Benchmark & Optimize

Duration: 30 min

After: Advanced Skill Systems lecture (Day 2, Block 3)

Prerequisites: Labs 2A-2B complete

Objective

Run a full **skill eval loop** (test set -> measure -> optimize -> retest), measure **run-to-run variance** so you know how many runs a number needs before you trust it, and run an **A/B check** of your skill against raw Claude - then decide: keep, optimize, or retire.

Deliverable (toolkit chain 7 of 12)

An eval test set at `.claude/skills/code-review/tests/eval-prompts.md`, a recorded **variance band**, plus a documented A/B verdict - the quality gate for your toolkit's skill.

START MARKER: `/clear done in business-dashboard.`

Fell behind? Run:

Set up this project with: (1) a code-review skill at `.claude/skills/code-review/SKILL.md` (frontmatter: name, description with review triggers, disable-model-invocation: false, allowed-tools: Read, Grep, Glob) producing RED/YELLOW/BLUE output, (2) a PostToolUse prettier hook and PreToolUse secret blocker registered in `.claude/settings.json`, (3) a human-invoked `/review` skill at `.claude/skills/review/SKILL.md` using `$ARGUMENTS`, (4) `CLAUDE.md` with project rules. Make scripts executable.

Exercise 1: Build the Eval Test Set (6 min)

Good skills trigger reliably. Prove it with data, not vibes.

Create an eval test set at `.claude/skills/code-review/tests/eval-prompts.md` with two sections:

SHOULD TRIGGER (true positives):

1. "Review the code in `src/routes/tasks.js`"
2. "Find bugs in this file"
3. "QA check on the task routes"
4. "Are there security issues in `server.js`?"
5. "Check this code for problems"

SHOULD NOT TRIGGER (true negatives):

1. "Write a new endpoint for user profiles"
2. "Explain how the Express router works"
3. "Run the tests"
4. "Refactor the dashboard for performance"
5. "Generate API documentation for `tasks.js`"

Exercise 2: Run the Eval (6 min)

`/clear`, then run at least 3 positives and 2 negatives. Score each:

Prompt	Fired? (RED/YELLOW/BLUE seen)	Pass ?
Review the code in <code>src/routes/tasks.js</code>	___	___
Find bugs in this file	___	___
QA check on the task routes	___	___
Write a new endpoint for user profiles	___ (should be NO)	___
Refactor the dashboard for performance	___ (should be NO)	___

Expected output marker: a score like 4/5. Perfect is rare - activation is probabilistic.

Watch for a conflict: keep `code-review` model-invoked for natural-language requests and `review` human-invoked for explicit `/review $ARGUMENTS` calls. Both should delegate to the same review checklist so there is one source of truth.

Exercise 3: Run Variance - Trust the Band, Not the Run (6 min)

That 4/5 you just wrote down - is it real? One run is an anecdote. Pick your *shakiest* SHOULD-TRIGGER prompt (the one that barely fired, or the "Find bugs in this file" style vague one) and run it **5 times**, with `/clear` between runs:

Run	Fired? (Y/N)
1	___
2	___
3	___
4	___
5	___

Record the band: `fired X/5`. Now write both numbers into `eval-prompts.md`:

Append a "Variance" section to `.claude/skills/code-review/tests/eval-prompts.md` recording: the prompt tested, the 5-run result (X/5), and this rule of thumb: "No conclusion from a single run. Five runs minimum before trusting a trigger rate. A change smaller than the run-to-run spread is noise, not signal."

Expected output marker: most people see 3/5-5/5 on a vague prompt - *the same skill, zero changes*. That spread is your noise floor: any "improvement" smaller than it is a coin flip.

Rule of thumb: 5 runs minimum before you trust an eval number; more if the spread is wide. Apply it to everything in Exercise 4.

Exercise 4: A/B Check - Skill vs Raw Claude (8 min)

Does the skill actually improve output, or has the model caught up?

Step 1 - baseline (skill OFF):

Rename `.claude/skills/code-review/SKILL.md` to `SKILL.md.bak`

/clear, then:

```
Review src/routes/tasks.js for issues
```

Save the output (copy to a scratch file).

Step 2 - skill ON:

```
Rename .claude/skills/code-review/SKILL.md.bak back to SKILL.md
```

/clear, then the SAME prompt.

Step 3 - compare:

Criteria	Without skill	With skill
Structured RED/YELLOW/BLUE output?		
Used the project checklist?		
Cited project-specific conventions?		
Consistent format on a rerun?		

Verdict - read it against your Exercise 3 band:

Result	Action
Skill wins clearly (gap bigger than your band)	Keep
Skill wins slightly (gap inside your band)	No conclusion - rerun more, or call it a tie
Raw Claude just as good	Consider retiring - the model caught up

Two skill types: capability uplift (teaches something new - keeps its edge) vs workflow/preference (enforces YOUR format - models catch up). Yours is a workflow skill: its value is consistency, not capability.

Exercise 5: Optimize the Description (3 min)

Feed real failures back into the description:

```
Read my code-review skill at .claude/skills/code-review/SKILL.md.
Based on these eval results: [paste your table - which fired, which shouldn't have]
Suggest an improved description: sharper trigger phrases, tighter negative boundaries,
clearer separation from documentation and refactoring tasks. Update only the description -
keep the workflow unchanged.
```

/clear, rerun the failed prompts. **Expected output marker:** previously-failing prompts now behave - and if the improvement is smaller than your band, rerun before you celebrate.

Exercise 6: Hygiene Check (1 min)

```
/doctor
```

Expected output marker: /doctor reports on your setup, including unused skills. **Read only the unused-skill section and ignore the rest** - the full report is long, personal to your machine, and mostly irrelevant here. If code-review shows unused, your eval and doctor disagree: trust the eval, but note the drift.

Optional step. On some builds this runs a lengthy sweep. If it takes more than a minute, cancel it and move on - the eval you just ran is the real evidence.

FINISH MARKER - Success Checklist

- ☐ Eval test set exists with 5 positives + 5 negatives
- ☐ At least 5 prompts scored; trigger rate recorded
- ☐ Variance band recorded from 5 runs of one prompt (X/5 + rule of thumb in the file)
- ☐ A/B run completed with the comparison table filled in
- ☐ Keep/optimize/retire verdict written down - judged against the band, not a single run
- ☐ Description optimized and failed prompts retested
- ☐ (Optional) /doctor's unused-skill section reviewed

If Stuck

Problem	Recovery
Nothing triggers after rename	You forgot to <code>/clear</code> after restoring SKILL.md
Both command and skill fire	Make <code>/review</code> delegate to the skill (Exercise 2 note)
A/B outputs look identical	Your rules match model defaults - add project-specific checks the model can't guess
5 runs won't fit the timebox	Run 3 and note the band is provisional - never run 1
Eval results vary run to run	That IS Exercise 3's lesson - report the band, not the point

Debrief Questions

1. Why rerun the eval set after every model update?
2. Your skill "improved" from 6/8 to 7/8 after a description tweak - what do you check before believing it?
3. Capability uplift vs workflow skill - which survives model progress, and which is yours?
4. What signal would make you retire a skill rather than optimize it?